

Phishing Email Detection & SIEM Pipeline

Case Study: Equity Bank Kenya Targeted Phishing Simulation

SOC Analysis · Threat Detection Engineering · SIEM Integration

Leonard N

Cybersecurity Portfolio

1. Project Overview

This project simulates and detects phishing attacks targeting users of Equity Bank Kenya. The system is a complete end-to-end SOC pipeline that ingests raw email samples, extracts indicators of compromise, applies multi-layer heuristic analysis, and forwards structured events to a Splunk SIEM for detection and investigation.

The tool classifies emails as Legitimate, Suspicious, or Malicious, with a focus on banking impersonation attacks — including fake Equity Bank login alerts, credential harvesting links, and lookalike domains engineered to deceive customers.

2. Detection Pipeline

Each email is processed through a structured sequence of extraction, validation, and scoring stages before producing a verdict and shipping the event as structured JSON to Splunk.

Email Sample (.eml) — Raw phishing email file provided as input to the analyser.

Python Email Parser — Extracts headers (From, Return-Path, Subject), body content, and embedded links using the email and BytesParser libraries.

Security Checks — SPF validation · DKIM signature check · Domain extraction · Lookalike domain detection · Phishing keyword analysis · Suspicious link detection.

Risk Scoring Engine — Aggregates signals into a numeric score (0–10). High scores indicate malicious intent.

Verdict & JSON Logging — Assigns CLEAN / SUSPICIOUS / MALICIOUS verdict and serialises all findings to output.json for SIEM ingestion.

Splunk SIEM Ingestion — Structured JSON ingested via Splunk's Add Data workflow, enabling detection rules and investigation queries.

3. Tool Execution — phishing_tool.py

The screenshot below captures `phishing_tool.py` running against the sample email `samplesequity case1.eml`. The tool parses the message, applies all detection checks, and outputs a structured verdict. The terminal output confirms a risk score of 8/10 and a verdict of **MALICIOUS**, with flags for Lookalike domain, SPF fail, Phishing language, and a Suspicious login link.

```

1 import email
2 from email import policy
3 from email.parser import BytesParser
4 import dns.resolver
5 import requests
6 import re
7 import os
8 import json
9
10 VI_API_KEY = "YOUR_API_KEY"
11 SAMPLES_DIR = "samples/"
12 OUTPUT_FILE = "output.json"
13
14 def parse_email(file_path):
15     with open(file_path, 'rb') as f:
16         msg = BytesParser(policy=policy.default).parse(f)
17
18     return {
19         "from": msg.get("From"),
20         "subject": msg.get("Subject"),
21         "return_path": msg.get("Return-Path"),
22         "headers": dict(msg.items()),
23         "body": get_body(msg)
24     }
25
26 def get_body(msg):
27     try:
28         return msg.get_body(preferencelist=('html', 'plain')).get_content()
29     except:
30         return ""
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

PS C:\Users\USER\OneDrive\Desktop\cybersecurity-portfolio> python phishing_tool.py
--- Analyzing: samplesequity case1.eml ---
From: Equity Bank Alerts <alerts@equitybank-secure.co.ke>
Subject: Action Required: Unusual Activity Detected
Result: {'score': 8, 'flags': ['Lookalike domain', 'SPF fail', 'Phishing language', 'Suspicious link: https://equitybank-verification.co.ke/login'], 'verdict': 'MALICIOUS', 'links': ['https://equitybank-verification.co.ke/login']}
PS C:\Users\USER\OneDrive\Desktop\cybersecurity-portfolio>

```

Figure 1 — `phishing_tool.py` executing against `samplesequity case1.eml` in VS Code terminal

Key Detection Signals Identified

- Lookalike domain — `equitybank-verification.co.ke` crafted to impersonate the legitimate bank domain
- SPF failure — the sending address does not pass Sender Policy Framework validation
- Phishing language — subject line uses urgency cues to drive victim action
- Suspicious link — external login URL pointing to the attacker-controlled credential harvesting page

4. Splunk SIEM — Log Ingestion

After the tool produces output.json, the structured event is ingested into Splunk Enterprise using the Add Data workflow. Splunk automatically detects the JSON source type and parses each field — including analysis flags, verdict, score, sender metadata, and the phishing link — making every attribute searchable and alertable within the SIEM.

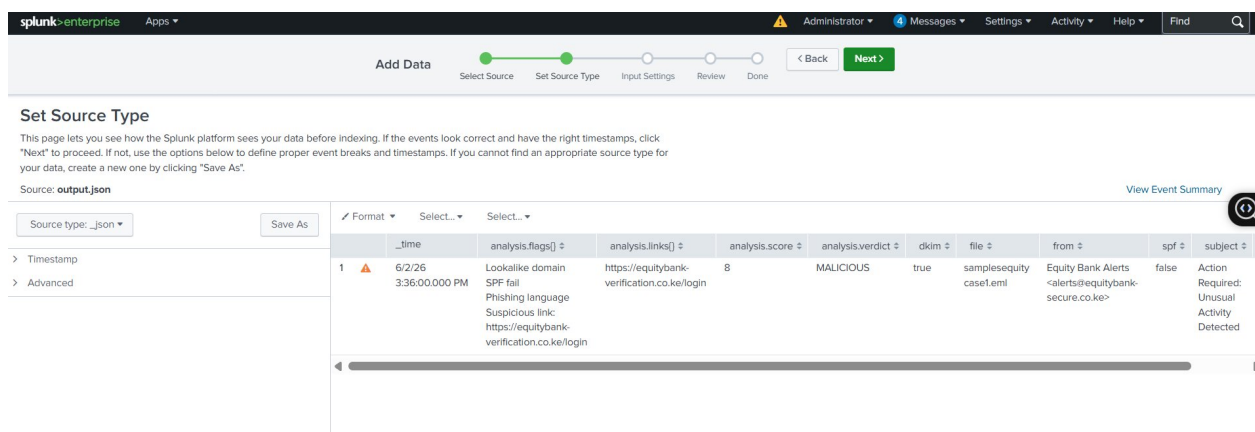


Figure 2 — Splunk Set Source Type screen showing parsed event fields from output.json

Parsed Fields in Splunk

- **analysis.flags** — Lookalike domain, SPF fail, Phishing language, Suspicious link
- **analysis.links** — Extracted URL: https://equitybank-verification.co.ke/login
- **analysis.score** — Risk score: 8
- **analysis.verdict** — MALICIOUS
- **from / spf / dkim** — Sender identity and authentication fields
- **subject** — Action Required: Unusual Activity Detected

5. Splunk Detection Queries (SPL)

Visibility Query — SOC Investigation View

This SPL query gives SOC analysts a broad view of all ingested email events. It extracts all structured fields and presents them in a tabular format for manual triage and risk review.

```
source="output.json" sourcetype="_json"
| spath
| table file from subject spf dkim analysis.verdict analysis.score analysis.flags
```

file	from	subject	spf	dkim	analysis.verdict	analysis.score	analysis.flags
sampleequity case1.eml	Equity Bank Alerts <alerts@equitybank-secure.co.ke>	Action Required: Unusual Activity Detected	false	true	MALICIOUS	8	
sampleequity case1.eml	Equity Bank Alerts <alerts@equitybank-secure.co.ke>	Action Required: Unusual Activity Detected	false	true	MALICIOUS	8	

Figure 3 — Visibility SPL query returning email metadata, verdict, score, and detection flags

Detection Query — Security Alert Rule

This SPL query filters exclusively for MALICIOUS verdicts, serving as the core detection rule. It can be saved as a scheduled alert to trigger incident response workflows automatically.

```
source="output.json" sourcetype="_json"
| spath
| search analysis.verdict="MALICIOUS"
| table file from subject analysis.score analysis.flags
```

file	from	subject	analysis.score	analysis.flags
sampleequity case1.eml	Equity Bank Alerts <alerts@equitybank-secure.co.ke>	Action Required: Unusual Activity Detected	8	
sampleequity case1.eml	Equity Bank Alerts <alerts@equitybank-secure.co.ke>	Action Required: Unusual Activity Detected	8	

Figure 4 — Detection SPL query isolating MALICIOUS emails with score and flag details

6. Attack Scenario & Outcome

The simulated attack targeted Equity Bank Kenya customers. The attacker sent a fraudulent security alert email impersonating the bank's official communications team, directing victims to a lookalike login page designed to harvest banking credentials.

Attack Flow

- Attacker crafts email from *alerts@equitybank-secure.co.ke* — a lookalike domain
- Subject line creates urgency: *Action Required: Unusual Activity Detected*
- Body contains a credential harvesting link pointing to the attacker's login page
- Victim clicks link and submits banking credentials to attacker-controlled infrastructure

Detection Outcome

- Spoofed banking email successfully detected and classified as MALICIOUS
- Risk score assigned: 8 out of 10
- Four phishing indicators flagged automatically
- Event ingested and fully structured in Splunk for SIEM-based alerting
- Detection query ready for scheduling as a real-time alert rule

7. Skills Demonstrated

Email Forensics — Parsing raw .eml files, extracting headers, body, and embedded links for forensic analysis.

SOC Analysis — Applying structured triage methodology to classify email threat level from extracted indicators.

Threat Detection Engineering — Designing multi-signal heuristic rules — SPF, DKIM, domain lookalike, keyword, and link analysis.

SIEM Integration (Splunk) — Ingesting structured JSON into Splunk, configuring source types, and validating field extraction.

SPL Query Development — Writing visibility and detection queries for investigation and automated alerting workflows.

Security Automation — Building an end-to-end Python pipeline replacing manual analysis with repeatable, auditable logic.

This project demonstrates a production-style SOC pipeline — from raw email ingestion through automated threat classification to SIEM-ready structured logging and detection query development.